19

BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

İlişkisel (SQL) vs NoSQL: ACID vs BASE ve Doğru Veri Modeli Seçimi

SQL vs NoSQL: ACID vs BASE & Picking the Right Data Store

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ SQL (RDBMS) - İLİŞKİSEL ]
• Örnek: PostgreSQL, MySQL
• Yapı: Katı Şema, Tablolar, FK
• Felsefe: ACID (Güçlü Tutarlılık)
• İlişkiler: JOIN işlemleri desteklenir
• Ölçekleme: Dikey (Scale-Up) odaklı

[ NoSQL - DAĞITIK DOKÜMAN / ANAHTAR ]
• Örnek: MongoDB, Cassandra, DynamoDB
• Yapı: Esnek Şema, JSON, Key-Value
• Felsefe: BASE (Nihai Tutarlılık)
• İlişkiler: Denormalize (Veri kopyalanır)
• Ölçekleme: Doğal Yatay (Scale-Out)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

SQL veritabanı, her satırı ve sütunu cetvelle çizilmiş, katı kuralları olan bir devlet tapu sicil dairesi gibidir: Her şey kanunlara (ACID) uyar; iki tapu birbiriyle çakışamaz (JOIN), ama defter büyüdükçe binaya yeni kat çıkmak (Dikey ölçekleme) zorunda kalırsınız. NoSQL ise devasa bir kargo aktarma merkezindeki pratik plastik koliler gibidir (BASE): Her kolinin içine istediğiniz şekilde eşya koyarsınız (esnek JSON şeması); 1 milyon koli gelirse yan tarafa 100 tane yeni depo inşa edip (Yatay ölçekleme) kolileri depolara fırlatırsınız!

🇬🇧 ENGLISH (ELI5)

A relational SQL database is a notarized government land registry: strictly structured tables, foreign keys, and absolute legal validity (ACID guarantees). Everything cross-references seamlessly via JOINs, but expanding means building a taller skyscraper (vertical scaling). NoSQL is an automated Amazon fulfillment sorting warehouse: shipping containers packed with self-contained JSON records (BASE philosophy). When traffic explodes, you don't build taller; you roll out 100 new commodity warehouse bays (horizontal scale-out)!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) SQL (ACID): Atomicity, Consistency, Isolation, Durability. Finansal işlemler, e-ticaret ödemeleri ve karmaşık ilişkisel sorgular (JOIN) için vazgeçilmezdir. Zorluğu: Milyarlarca satıra ulaştığında sharding ve cross-node join yapmak aşırı zordur. 2) NoSQL (BASE): Basically Available, Soft state, Eventual consistency. Şemasızdır (Schema-on-read); yatayda onlarca sunucuya yayılmak üzere tasarlanmıştır. Veriler sorgu desenine göre denormalize edilir (JOIN yapılmaz, veriler iç içe gömülür). Seçim Kriteri: Veriniz yüksek ilişkisel mi, yoksa saniyede yüzbinlerce okuma/yazma gerektiren analitik/akış verisi mi?

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) SQL (ACID): Strict schemas with normalized relations, foreign key constraints, and transactional isolation. Essential for financial ledgers, transactional checkout baskets, and complex relational graphs. Scaling ceiling: Cross-shard joins are computationally prohibitive and multi-master clustering introduces heavy consensus locking. 2) NoSQL (BASE): Basically Available, Soft State, Eventual Consistency. Schemaless flexibility (schema-on-read) built ground-up for horizontal sharding. Documents denormalize data to prioritize O(1) primary key queries over cross-table joins.

⚡ Modern Java 25 Uygulaması (Loom & Records) / ⚡ Java 25 Implementation

```
// Java 25: SQL (Normalized) vs NoSQL (Denormalized) Veri Modeli
public class DataModelComparison {

    // 1. SQL Tarzı: Normalize Edilmiş İlişkisel Yapı (Ayrı Tablolar + ID Referansı)
    public record SqlOrder(long orderId, long customerId, java.math.BigDecimal totalAmount) {}
    public record SqlOrderItem(long itemId, long orderId, String productName, int quantity) {}

    // 2. NoSQL Tarzı: Denormalize Edilmiş Doküman (Tek Parça, Sıfır JOIN!)
    public record NoSqlOrderDocument(
        String orderId,
        String customerName,
        String customerEmail,
        java.util.List<OrderItemDoc> items,
        java.math.BigDecimal totalAmount
    ) {
        public record OrderItemDoc(String product, int qty, java.math.BigDecimal price) {}
    }
}

void main() {
    var item = new DataModelComparison.NoSqlOrderDocument.OrderItemDoc("MacBook Pro M4", 1, new
java.math.BigDecimal("2500"));
    var doc = new DataModelComparison.NoSqlOrderDocument("ord_101", "Ali Bozkurt", "ali@test.com",
java.util.List.of(item), new java.math.BigDecimal("2500"));
    IO.println("NoSQL Tekil Doküman Oluşturuldu: ID=" + doc.orderId() + ", Ürün=" +
doc.items().getFirst().product());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı / ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçının 'Hangi veritabanını seçersin?' sorusuna hemen 'MongoDB!' veya 'PostgreSQL!' diye atlamayın. Önce veri erişim modelini (Access Pattern) analiz edin: 'Veri ilişkisel mi? Güçlü ACID gerekiyor mu? Yoksa yüksek hacimli zaman serisi veya anahtar-değer erişimi mi var?' diyerek gerekçenizi temellendirin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never blurt out 'I'll use MongoDB!' before analyzing the query access patterns. State the decision matrix: 'Because our workload requires complex relational multi-table transactions, I recommend PostgreSQL' or 'Because our writes hit 500K QPS with simple key-value reads, Cassandra is optimal.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar / 🎯 Key Takeaways for the Interview

- Finansal tutarlılık ve zengin JOIN sorguları için PostgreSQL/MySQL tercih edilir.
- Yüksek hacimli, şeması değişken ve yatay ölçeklenebilir veriler için NoSQL seçilir.
- Modern mimarilerde Hibrit (Polyglot Persistence) yaklaşım kullanılır (ör. Siparişler SQL'de, Akış Cassandra'da).

- RDBMS (PostgreSQL/MySQL) remains optimal for transactional ACID and relational queries.
- NoSQL fits high-throughput, denormalized, horizontally scalable workloads.
- Enterprise architectures adopt Polyglot Persistence: SQL for orders, NoSQL for feeds.